

Looking at the code you pasted, the original developer kept the prompts in text files but dynamically injected the `PROFILE` , `JOB DESCRIPTION` , and `JSON SCHEMAS` using Python's `f-strings` on every single API call. That uses a ton of unnecessary tokens. We can completely refactor this into the `start_chat()` architecture. By doing this, we load your massive `PROFILE.txt` and the system rules just once. Then, the AI remembers it for the suitability check, the resume, and the cover letter. Here is how you organize your folders and files to pull this off cleanly.

1. The Directory Structure

Create this layout. It separates your personal data, your instructions, and your Python logic.

Plaintext

```
cv_pipeline/  
├── context/  
│   ├── system_instruction.md  
│   └── cindy_profile.txt  
├── prompts/  
│   ├── 1_check.md  
│   ├── 2_resume.md  
│   └── 3_coverletter.md  
└── automate_v2.py
```

2. The Context Files (Loaded once at startup)

`context/cindy_profile.txt`

Just paste the exact contents of your `PROFILE.txt` here. Don't change a thing.

`context/system_instruction.md`

This is the anchor. We merge the core rules from your different prompts into one master persona.

Markdown

You are an expert HR Analyst, ATS Machine, and Career Strategist.
Your job is to evaluate job descriptions and perfectly tailor my profile to match them.

CRITICAL RULES:

- Never hallucinate requirements, skills, or experience.
- Only use information from the provided Master Profile.
- Be strictly objective and results-focused.
- Do not output any markdown, prose, or conversational filler outside of the requested JSON formats.

Here is my Master Profile:

{{PROFILE_DATA}}

3. The Prompt Files (Sent in sequence)

Notice how Phase 2 and Phase 3 do not need to see the profile or the job description again. They just read the chat history.

prompts/1_check.md

Markdown

Evaluate this job description against my profile. Be strict, realistic, and practical.
If there are major blockers (like missing a mandatory PhD, specific tech stack, or language), mark suitability as false.

Here is the Job Description:

{{JOB_DESCRIPTION}}

Output strictly in this JSON schema:

```
{
  "match_score": 0,
  "is_suitable": true,
  "visa_sponsorship": "NO SPONSORSHIP EXPLICITLY | NO TALK ABOUT SPONSORSHIP | SPONSORSHIP/RELOCATION IS CONSIDERED | SPONSORSHIP IS G
  "language_fit": "string (comma separated list of required languages)",
  "missing_requirements": ["string"],
```

```
"mandatory_documents": ["string"],
"cover_letter_required": true,
"additional_answers": [{"question": "string", "answer": "string"}]
}
```

prompts/2_resume.md

Markdown

Based on the job description you just analyzed, generate a tailored CV.

- Keep bullets tight (80-100 characters). Match duties with quantifiable outcomes using my existing data.
- Merge the Wilmar research into the SDLC position if the job isn't research-focused.
- Budget: Exactly 1 A4 page (max 40 lines, 500-570 words). Dedicate the most volume to the recent Wilmar role.
- Do not generate placeholder text.

Output strictly in this JSON schema:

```
{
  "summary": ["string"],
  "tech": [{"label": "string", "value": "string"}],
  "skills": [{"label": "string", "value": "string"}],
  "experience": [
    {
      "title": "string",
      "company": "string",
      "location": "string",
      "time": "string",
      "bullets": ["string"],
      "projects": ["string"]
    }
  ],
  "projects": [
    {
```

```
    "title": "string",
    "bullets": ["string"]
  }
],
"education": {
  "focus": "string",
  "thesis": "string"
},
"awards": ["string"]
}
```

prompts/3_coverletter.md

Markdown

Now, write a personalized cover letter for this same job.

- Connect my experience and open-source projects directly to the job requirements.
- Mention I am willing to relocate and eligible for the EU Blue Card (4+ years exp, EMJMD Master's).
- Keep it 3-4 paragraphs, around 200 words.
- Tone: Human, casual enough for conversation, formal enough for work. Vary sentence lengths. Avoid AI buzzwords.
- Do not start with "I've been following..."

Output strictly in this JSON schema:

```
{
  "job_title": "string",
  "hiring_manager_name": "string",
  "company_name": "string",
  "company_address": "string",
  "company_location": "string",
  "letter_date": "string",
}
```

```
"body": ["paragraph 1", "paragraph 2", "paragraph 3"]
}
```

4. The Python Runner (automate.py)

This replaces that massive 450-line automate.py. All the heavy lifting is handled by the text files and the Gemini API context window.

Python

```
import os
import json
import google.generativeai as genai

# Helper to load and inject variables into markdown
def load_prompt(filepath, **kwargs):
    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read()
    for key, val in kwargs.items():
        content = content.replace(f"{{{key}}}", val)
    return content

def main():
    # 1. Get the Job Description from the user
    job_desc = input("Paste the Job Description here (press Enter to submit): ")
    if not job_desc:
        print("No job description provided. Exiting.")
        return

    # 2. Setup Gemini and Context
    api_key = os.environ.get("GEMINI_API_KEY")
    genai.configure(api_key=api_key)
```

```

profile_text = load_prompt("context/cindy_profile.txt")
system_instruction = load_prompt("context/system_instruction.md", PROFILE_DATA=profile_text)

# Initialize the model with the system persona
model = genai.GenerativeModel(
    model_name="gemini-1.5-pro",
    system_instruction=system_instruction,
    generation_config={"response_mime_type": "application/json"}
)

# Start the persistent chat session
chat = model.start_chat(history=[])

# -----
# PHASE 1: JOB CHECK
# -----
print("\n🔍 Analyzing job suitability...")
prompt_check = load_prompt("prompts/1_check.md", JOB_DESCRIPTION=job_desc)
res_check = chat.send_message(prompt_check)
check_data = json.loads(res_check.text)

print(f"Match Score: {check_data.get('match_score')}%")
if not check_data.get('is_suitable'):
    print("🚫 Job is not suitable based on blockers. Stopping pipeline.")
    return

# -----
# PHASE 2: GENERATE CV
# -----
print("\n📄 Generating tailored CV data...")
prompt_cv = load_prompt("prompts/2_resume.md")
res_cv = chat.send_message(prompt_cv)
cv_data = json.loads(res_cv.text)

```

```

with open("data_cv.json", "w", encoding="utf-8") as f:
    json.dump(cv_data, f, indent=2)
print("✅ Saved data_cv.json")

# -----
# PHASE 3: GENERATE COVER LETTER
# -----
if check_data.get("cover_letter_required"):
    print("\n📄 Generating tailored Cover Letter...")
    prompt_cl = load_prompt("prompts/3_coverletter.md")
    res_cl = chat.send_message(prompt_cl)
    cl_data = json.loads(res_cl.text)

    with open("data_cl.json", "w", encoding="utf-8") as f:
        json.dump(cl_data, f, indent=2)
    print("✅ Saved data_cl.json")
else:
    print("\n🔊 Cover letter not mandatory. Skipping.")

print("\n🎉 Pipeline complete. You can now pass the JSON files to your docx generators.")

if __name__ == "__main__":
    main()

```

That is it. You just run this script. It reads your prompts, hits the API, remembers the context, and spits out the exact JSON files your original Word document generators need to build the final `.docx` files.